

ELECTRONIC SYSTEMS 1

Laboratory – Preparation and Tasks



Reinhard Feger, Werner Haselmayr

Winter Term 2025/2026

Institute for Communications Engineering and RF-Systems, nthfs.jku.at

Outline

Goals of the Laboratory

Preparation Part 1 (Homework)

- Installation

- GNU Radio Example

Preparation Part 2 (at University)

Control of CC1101 Chip

Tasks

GOALS OF THE LABORATORY



Overview

- Implementation and Analysis of a wireless communication link using Software Defined Radio (SDR) and the CC1101 transceiver chip
 - SDR will be used as receiver which downconverts the received RF signal and allows for further signal processing (e.g., filtering)
 - The CC1101 transceiver chip will be used as transmitter and receiver and will be controlled by a microcontroller-board (eLITe-board)
- The laboratory includes the following task
 - Analysis of the time- and frequency-behavior of 2-FSK, 4-FSK and GFSK
 - Implementation of a noncoherent receiver for 2-FSK using SDR
 - Analysis of the limits of the implemented wireless link in terms of baudrate, transmit power and distance

PREPARATION PART 1 (HOMEWORK)



Outline

Goals of the Laboratory

Preparation Part 1 (Homework)

Installation

GNU Radio Example

Preparation Part 2 (at University)

Control of CC1101 Chip

Tasks

Requirements

- Laptop with Windows 10 (64bit) or Linux
 - MacOS (with Homebrew) is likely to work, but untested
 - Android, Chrome OS, etc. is not supported
- At least one USB 3.0/3.1 (USB 3.2 Gen 1/USB 3.2 Gen 2) port (Type-A) and an additional Type-A USB (can be a slower one) port
- The following step-by-step installation guideline applies to Windows only¹

¹Please contact the instructor if you want to use a different operating system.

Python Environment

- Install Radioconda from
`https://github.com/ryanvolz/radioconda#download`
- If you have an existing `conda` installation: See
`https://github.com/ryanvolz/radioconda#install` where it is explained
how to skip the installer and create a new environment in your existing `conda`
installation

Micropython IDE – Thonny

- Allows direct connection with the Micropython interpreter running on the eLITE-board (file transfer, text output, plots, ...)
- Either open the Radioconda-prompt from the Windows start menu or activate your radioconda environment in your existing conda prompt²
- To install Thonny (Thonny is not available in any conda repository) run:

○ `pip install thonny`

```
1 from machine import Pin
2 from time import sleep
3
4 led = Pin(2, Pin.OUT)
5
6 while True:
7     led.value(not led.value())
8     sleep(0.5)
```



- Start `thonny` from the command line and **unchecked** *Allow only single Thonny instance* under *Tools* → *Options* → *General*

²by running `conda activate radioconda`

GNU Radio Companion

- GNU Radio is a software development toolkit, which provides signal processing blocks to implement Software Defined Radio (SDR) systems
- GNU Radio Companion is a graphical user interface for GNU Radio
 - Can be used for various SDRs
 - RX only SDRs³ start around 20€ (if you want to experiment yourself)
- Universal software radio peripherals (USRPs) are used as SDR device
- To start GNU Radio Companion
 - use the Start menu or
 - run `gnuradio-companion` from the command line (in the correct conda environment)

³https://de.wikipedia.org/wiki/Software_Defined_Radio#RTLSDR

USRP Firmware

- Please run the following download at home:
- Open your conda prompt (depending on the type of installation: activate the `radioconda` environment), and run `uhd_images_downloader`

Outline

Goals of the Laboratory

Preparation Part 1 (Homework)

Installation

GNU Radio Example

Preparation Part 2 (at University)

Control of CC1101 Chip

Tasks

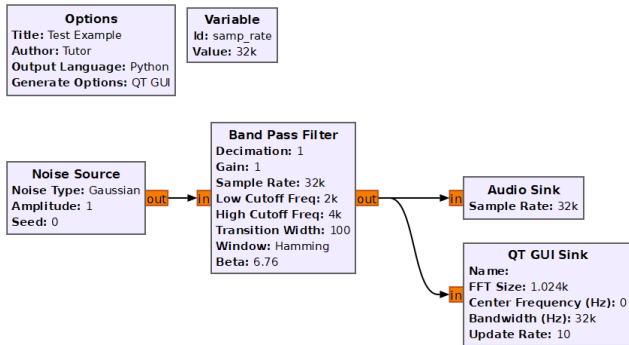
GNU Radio Example – Overview (1/2)

- Please prepare the following example to become familiar with GNU Radio Companion
 - The example will be checked at the beginning of the laboratory and is mandatory for passing the course
 - If you have any questions, please post your problem in the Moodle-Forum (<https://moodle.jku.at/mod/forum/view.php?id=12015031>)
- A brief description on how to build this examples can be found on the next slides⁴

⁴Please refer to the detailed description of various GNU Radio Companion building blocks in the document “Laboratory – GNU Radio Companion and Python”, which is available in Moodle.

GNU Radio Example – Overview (2/2)

- The aim of the example is to show how signal generation, filtering, signal analysis, etc.
- The flow graph of the example looks as follows

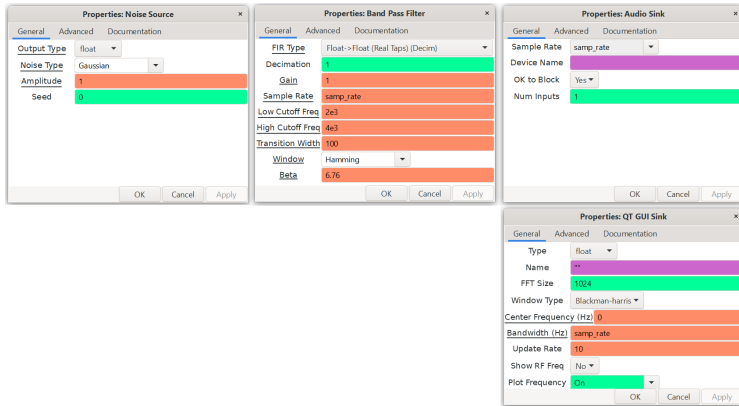


GNU Radio Example – Setup

- Open your conda prompt (depending on the type of installation: activate the `radioconda` environment)
- Start GNU Radio Companion (`gnuradio-companion`)
- Insert all blocks (*Options*, *Noise Source*, ...) shown on the previous slide
 - Insert blocks by drag and drop from the *Library* on the right
 - Use `Ctrl+F` to search for blocks
- Set the ID of the *Options*-block to "test_example" and save the file to your folder
 - Double-click on the blocks enables the configuration
 - Default ID must be changed to be able to run the program
- Connect the blocks by drag and drop on the output and input ports

GNU Radio Example – Block Configuration

- Set the configuration of all blocks as shown below
 - Double-click on the blocks opens the menu



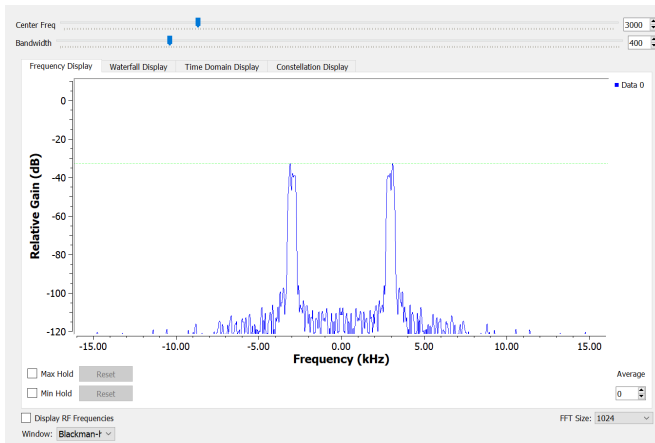
GNU Radio Example – Testing

- Try different settings⁵
 - Change the cutoff frequency in the *Band Pass Filter*-block when the program is running. What do you observe?
 - Use the *QT GUI Range*-block to be able to change the center frequency and the bandwidth using a slider. This feature will be helpful during the laboratory.
 - Insert different source blocks (Noise Source, Signal Source or Audio Source to use your microphone) and investigate the behavior of the bandpass filter.
- **Hint:** Select a block and press *E* or *D* to **enable** or **disable** the block temporarily

⁵Please feel free to investigate more than the things listed here

GNU Radio Example – Possible Output

- A possible output of the *QT GUI Sink*-block could look as follows

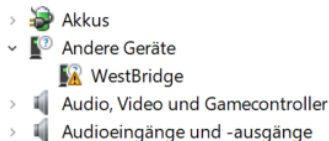


PREPARATION PART 2 (AT UNIVERSITY)



USRP Installation (1/4)

1. Plug in the USRP's power supply and connect it to the PC using an USB 3.0 port
2. Open the *Device Manager*
 - Type *Device Manager* in the search field of the taskbar **OR**
 - Right-click on the Windows-icon and select *Device Manager* from the drop-down menu
3. A new entry with the name *WestBridge* should appear under *Other Devices*. This indicates that the device is connected but a suitable driver has not been installed yet.



USRP Installation (2/4)

4. Follow the instructions from
`https://github.com/ryanvolz/radioconda#windows-users-6`
5. and make sure to use the “Zadig” approach: `https://github.com/ryanvolz/radioconda#installing-the-winusb-driver-with-zadig`

USRP Installation (3/4)

6. The *Device Manager* should automatically update the device list and “WestBridge” should appear without the yellow exclamation mark (it could also have a different name).
7. Open your conda prompt (depending on the type of installation: activate the `radioconda` environment) and launch `uhd_find_devices`
 - This program loads the firmware image `usrp_b200_fw.hex` onto the USRP
 - In the best case the USRP will be found immediately

```
C:\Users\Acer>uhd_find_devices
[INFO] [UHD] Win32; Microsoft Visual C++ version 14.1; Boost_107000; UHD_4.0.0.0-release
-----
-- UHD Device 0
-----
Device Address:
  serial: 30B56ED
  name: NI2901
```

USRP Installation, Optional Step (4/4)

8. The USRP device with the new firmware might appear once again in the section *Other Devices* in the *Device Manager* with an exclamation mark. In this case the device cannot be found:

```
C:\Users\Acer>uhd_find_devices
[INFO] [UHD] Win32; Microsoft Visual C++ version 14.1; Boost_107000; UHD_4.0.0.0-release
[INFO] [B200] Loading firmware image: C:\Program Files\PothosSDR\share\uhd\images\usrp_b200_fw.hex...
No UHD Devices Found
```

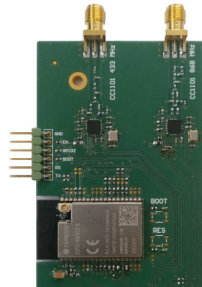
9. If this is the case: please **repeat the driver installation** according to the previous step **3** (don't disconnect the USRP in between).
10. After the successful driver installation launch `uhd_find_devices` again. The result should be similar to the screenshot of slide 3/4.

CONTROL OF CC1101 CHIP



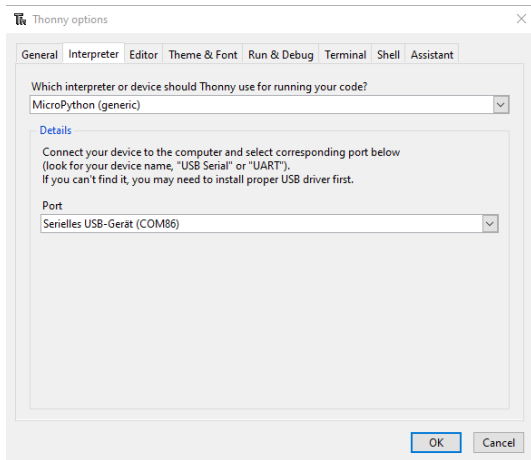
Hardware

- The CC1101 transceiver chip will be controlled via the eLITE-board (microcontroller-board based on a STM32 microcontroller) through a serial peripheral interface (SPI)
- Two CC1101 transceivers chips are mounted on an extension board (eLITE-connect) on the bottom of the eLITE-Board.



Software

- The eLITe-board is flashed with Micropython, which is a small Python interpreter running on the microcontroller
- Connect the eLITe-board to your PC via an USB-serial port
- Start Thonny (`thonny`) from the `radioconda` environment/command line and select *MicroPython (generic)* and the correct USB port under *Run* → *Select Interpreter...*



USB-Serial Connection Tips and Tricks

- **Problem:** Windows makes it difficult to find the correct USB port
Solution: Trial-and-error until the Thonny Shell shows something like the following

```
1  MicroPython v1.13-301-g8baa70eea-dirty on 2021-02-16; ELITEBOARD_H743XI with STM32H743
2  Type "help()" for more information.
```

- **Problem:** Driver is highly dependent on your configuration
Solution: Manual installation of the virtual serial port driver⁶ (see Moodle for a zip-file). Running “pybcdc.cmd” should automatically recognize your system configuration.

⁶<https://github.com/openmv/qt-creator/tree/c2633fc04949d5a0818000b2a26334be3e88dac5/share/qtcreator/drivers/pybcdc>

Thonny IDE

- Files can be stored locally and on the eLITE-board
- A small file explorer can be shown by selecting *Files* under *View*
- Single files from the PC can be executed on the eLITE-board
 - Modules are only found when they are available on the eLITE-board
 - Don't forget to also save your programs locally when you work on the eLITE-board

• Example: Blinking LED

```
1  from umachine import Pin
2  import time
3  import umachine
4  umachine.freq(int(50e6)) # reduce pwr. consump.
5
6  led_num = 1 # 1...9
7  seg1 = Pin('LED{}'.format(led_num), Pin.OUT)
8
9  while True:
10     seg1.on()
11     time.sleep(0.1)
12     seg1.off()
13     time.sleep(0.1)
```

Note: A running program can be stopped by simultaneously pressing `Ctrl+C` in Thonny's *Shell*-tab

Controlling CC1101 – SPI Connection

- Micropython allows to realize an SPI connection on arbitrary Pins
- Pins can be accessed with the *Pin*-class from the *machine*-module
- These *Pin*-objects can then be used to create a *SoftSPI*-object
- Example: A code snippet for accessing the Pins of the two CC1101 chips (eLITE-connect) is shown on the right

```
1  from umachine import Pin, SoftSPI
2  # ---- eLITE-connect
3  pCS433 = Pin(Pin.cpu.I10, Pin.OUT) #433
4  pCS433.on()
5  pGD00433 = Pin(Pin.cpu.I12, Pin.IN) #433
6  pGD02433 = Pin(Pin.cpu.I13, Pin.IN) #433
7
8  pCS868 = Pin(Pin.cpu.I4, Pin.OUT) #868
9  pCS868.on()
10 pGD00868 = Pin(Pin.cpu.I7, Pin.IN) #868
11 pGD02868 = Pin(Pin.cpu.I8, Pin.IN) #868
12
13 pSCK = Pin(Pin.cpu.K0, Pin.OUT)
14 pMOSI = Pin(Pin.cpu.J10, Pin.OUT)
15 pMISO = Pin(Pin.cpu.F8, Pin.IN)
16 # ---- eLITE-connect
17 spi = SoftSPI(
18     baudrate=int(1e5), polarity=0, phase=0, bits=8,
19     firstbit=SoftSPI.MSB, sck=pSCK, miso=pMISO, mosi=pMOSI
20 )
```

Controlling CC1101 – Available Function (1/2)

- There is no need to program the CC1101 registers bit-by-bit
- If you have a valid SPI connection and 2 synchronization pins you can create a CC1101 object and initialize it

```
1  from pycc1101 import TICC1101
2
3  CC1101_433 = TICC1101(spi, pCS=pCS433,
4                        pGD00=pGD00433,
5                        pGD02=pGD02433, debug=False
6                        )
7  CC1101_433.reset()
8  CC1101_433.selfTest()
9  CC1101_433.setDefaultValues()
```

Controlling CC1101 – Available Functions (2/2)

- Some useful functions for controlling the CC1101
-

```
1  CC1101_433.configureAddressFiltering("DISABLED")
2  # CC1101_433.configureAddressFiltering("ENABLED_NO_BROADCAST")
3  # CC1101_433.setFilteringAddress(0x9D)
4  CC1101_433.setChannel(11)  # Standard channel spacing approx. 200 kHz
5  CC1101_433.setBaud(1000)
6  CC1101_433.enWhiteData(False)
7  CC1101_433.enFEC(False)
8  CC1101_433.enCRC(False)
9  CC1101_433.setModulation("2-FSK")  # Possible values: 2-FSK, 4-FSK, GFSK, ASK
10 CC1101_433.setDevtnDefault()  # Alternative: CC1101_433.setDevtnLarge()
11 CC1101_433.setPacketMode("PKT_LEN_FIXED")
12 CC1101_433.setPktLen(2)  # Includes sync word
13 CC1101_433.setSyncWord("OFF")  # Default: 0xD391
14 # CC1101_433.setTXPower(paTableIdx)
15 # paTable in dBm: -30, -20, -15, -10, 0, 5, 7, 10
```

Controlling CC1101 – Sending and Receiving Data

- Sending data (after successful configuration):

```
1 data = [0xA5]
2
3 for j in range(1000):
4     print("send: {:d}".format(
5         j))
6     print(CC1101_433.sendData(
7         data))
8     time.sleep(0.1)
```

- Receiving data:

```
1 cnt = 0
2 while True:
3     data = CC1101_433.recvData()
4     if data:
5         dataStr = ""
6         dataHexStr = ""
7         for d in data:
8             dataStr += chr(d)
9             dataHexStr += hex(d)+' '
10        print("RX message {}: {}".format(cnt,
11            dataStr))
12        print("RX message hex: ", dataHexStr)
13        cnt += 1
14        print("RSSI: {}".format(CC1101_433.
15            getRSSI()))
```

TASKS

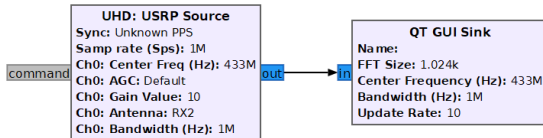


Overview

- **Task 0:** Carry out *Preparation Part 1/2* and test USRP functionality
- **Task 1:** Set up a receiver in GNU Radio Companion and try to receive the signal sent by the instructor's eLITE-board transmitter
- **Task 2:** Implement a transmitter using the eLITE-board
- **Task 3:** Set up a noncoherent receiver for 2-FSK in GNU Radio Companion
- **Task 4:** Implement a receiver using the eLITE-board and investigate the performance bounds
- Legal stuff (please do not exceed these limits!)
 - https://en.wikipedia.org/wiki/ISM_radio_band
 - https://en.wikipedia.org/wiki/Short-range_device#SRD860

Task 0

- Carry out preparation part 1 (at home) and part 2 (at university)
- Plug in the USRP's power supply and connect it to the PC via an USB 3.0 port
- Start GNU Radio Companion (`gnuradio-companion`)
- Place a *UHD: USRP Source*-block and configure it as shown on slides 11/12 (*Capture Signal from Hardware*) in the document `ElSys1LabGnuRadioPython.pdf`
 - Sample rate: 1 MHz
 - Center frequency: 433 MHz
- Check the USRP functionality through the following example



Task 1

- Set up a receiver in GNU Radio Companion and try to receive the signal sent by the instructor's eLITE-board transmitter. The transmitter settings are as follows:
 - $f_{\text{osc}} = 26 \text{ MHz}$
 - Modulation: 2-FSK
 - Baudrate: 1000 Baud
 - Sync word: 0x0F0F
 - Sent byte: 0xA5
 - (Frequency) Deviation: Exponent $E = 0b100$, mantissa $M = 0b111$
- **Hint:** Use the *Quadrature Demod*-block for the demodulation
- **Report**
 - Plot the received signal in the frequency-domain as well as the bit-stream in time-domain and interpret the results
 - Compare the bit-stream with the signal shown in the CC1101 data sheet in Fig. 2 on p. 42

Task 2

- Implement a transmitter using the eLITe-board
 - Use the channel assigned to you by the instructor (channel spacing 200 kHz)
 - Set the center frequency of the USRP source to your channel frequency
 - Set the frequency deviation as in Task 1
 - Use different modulation schemes: 2-FSK, 4-FSK, and GFSK
- Use the implementation from Task 1 as receiver and extend it by a symmetric *Low Pass Filter*-block to extract only your channel from the received signal
- **Hints:**
 - Use the source code presented on the slides 25-27 (*Controlling CC1101*) for the implementation
 - Make sure that the filter transition width is not too low
- **Report**
 - Plot the received signal for the different transmitter settings in the time- and frequency domain and interpret the results

Task 3

- Set up a noncoherent receiver for 2-FSK in GNU Radio Companion
- To implement an asymmetric filter (different for positive and negative frequencies) check slides “GNU Radio Companion – Filter” in `ElSys1LabGnuRadioPython.pdf`
- Use the implementation from Task 2 as transmitter
- **Hint:** A possible implementation of a noncoherent receiver can be found on the slides of Lecture 5
- **Report**
 - Block diagram of the implemented noncoherent receiver
 - Settings of the bandpass filters
 - Plot the received signal in time- and frequency-domain and interpret the results

Task 4

- Implement transmitter (use eLITe-board from Task 2) and receiver using a second eLITe-board
- Analyze the free-space behavior by observing received signal strength indicator (RSSI) values (converted to dBm according to chapter 17.3 in the datasheet) for varying
 - transmit power (at least three values) and
 - distance (at least four values)
- **Report**
 - Plot received power vs. transmit power and compare with the expected behavior
 - Plot received power vs. distance and compare to the Friis equation. The antenna gains are unknown, but the relative course should match (apart from the multipath behavior in our indoor scenario)

Task 5

- Implement ASK transmitters by setting `CC1101_433.setModulation("ASK")` and using the code snippets for “hard” bit transitions

```
1 CC1101_433.setPATable([0x0, 0xC0])
2 CC1101_433.setPARampingSteps(1)
```

and “smooth” bit transitions (see datasheet p. 59-61)

```
1 CC1101_433.setPATable([0x0, 0x0E, 0x1D, 0x34, 0x60, 0x84, 0xC8, 0xC0])
2 CC1101_433.setPARampingSteps(7)
```

- **Report**
 - Extract the pulse sequence from the received signal and plot it in a comparable way as in the other tasks